Practical 5: MiniBank Polymorphic Account Hierarchy

Problem Definition:

Convert Account to an abstract class with abstract methods interestRate() and canWithdraw(long amount); implement concrete subclasses SavingsAccount (min balance Rs. 1000, 4% interest), CurrentAccount (overdraft Rs. 5000), and FixedDepositAccount (locked funds).

Code:

```
class PartB {
    public static abstract class Account {
        private final String accountNumber;
        private String ownerName;
        private long balance;

        public Account(String accountNumber, String ownerName, long balance) {
            this.accountNumber = accountNumber;
            this.ownerName = ownerName;
            this.balance = balance;
        }

        public String getAccountNumber() {
            return accountNumber;
        }

        public String getOwnerName() {
            return ownerName;
        }

        public long getBalance() {
            return balance;
        }

        public void deposit(long amount) {
            if (amount > 0) {
                balance += amount;
                System.out.println("Deposited Rs. " + amount + " into " + accountNumber + " |
New Balance: Rs. " + balance);
            }
        }

        public boolean withdraw(long amount) {
            if (amount > 0 && canWithdraw(amount)) {
                balance -= amount;
                System.out.println("Withdrew Rs. " + amount + " from " + accountNumber + " |
Remaining Balance: Rs. " + balance);
                return true;
            } else {
                System.out.println("Withdrawal of Rs. " + amount + " failed on " + accountNumber
+ "! (Policy violation / Insufficient funds)");
                return false;
            }
        }

        public abstract double interestRate();
```

```
        public abstract boolean canWithdraw(long amount);

        public abstract String getAccountType();

        @Override
        public String toString() {
            return "[" + getAccountType() + "] " + accountNumber + " (" + ownerName + ") |
Balance: Rs. " + balance + " | Rate: " + interestRate() + "%";
        }
    }

    public static class SavingsAccount extends Account {
        private final long minBalance;

        public SavingsAccount(String accountNumber, String ownerName, long balance, long
minBalance) {
            super(accountNumber, ownerName, balance);
            this.minBalance = minBalance;
        }

        public long getMinBalance() {
            return minBalance;
        }

        @Override
        public double interestRate() {
            return 4.0;
        }

        @Override
        public boolean canWithdraw(long amount) {
            return (getBalance() - amount) >= minBalance;
        }

        @Override
        public String getAccountType() {
            return "Savings";
        }
    }

    public static class CurrentAccount extends Account {
        private final long overdraftLimit;

        public CurrentAccount(String accountNumber, String ownerName, long balance, long
overdraftLimit) {
            super(accountNumber, ownerName, balance);
            this.overdraftLimit = overdraftLimit;
        }

        public long getOverdraftLimit() {
            return overdraftLimit;
        }

        @Override
        public double interestRate() {
            return 0.0;
        }
```

```
        @Override
        public boolean canWithdraw(long amount) {
            return (getBalance() - amount) >= -overdraftLimit;
        }

        @Override
        public String getAccountType() {
            return "Current";
        }
    }

    public static class FixedDepositAccount extends Account {
        private int termMonths;

        public FixedDepositAccount(String accountNumber, String ownerName, long balance, int
termMonths) {
            super(accountNumber, ownerName, balance);
            this.termMonths = termMonths;
        }

        public int getTermMonths() {
            return termMonths;
        }

        @Override
        public double interestRate() {
            return 7.0;
        }

        @Override
        public boolean canWithdraw(long amount) {
            return false;
        }

        @Override
        public String getAccountType() {
            return "FixedDeposit";
        }
    }

    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 5 - Part B: MiniBank Polymorphic Account Hierarchy");
        System.out.println();
        Account[] accounts = {
            new SavingsAccount("AC0001", "Rudra Patel", 8000, 1000),
            new CurrentAccount("AC0002", "Aarav Sharma", 4000, 5000),
            new FixedDepositAccount("AC0003", "Diya Shah", 50000, 12)
        };

        System.out.println("MiniBank Account Subtypes & Polymorphic Rates:");
        for (Account acc : accounts) {
            System.out.println(acc);
            System.out.printf("  -> Verified Interest Rate: %.1f%%%n", acc.interestRate());

            if (acc instanceof SavingsAccount sa) {
                System.out.println("  -> [Savings Rule] Minimum mandatory balance: Rs. " +
```

```
sa.getMinBalance());
            } else if (acc instanceof CurrentAccount ca) {
                System.out.println("  -> [Current Rule] Approved Overdraft Limit: Rs. " +
ca.getOverdraftLimit());
            } else if (acc instanceof FixedDepositAccount fda) {
                System.out.println("  -> [FD Rule] Term Lock Period: " + fda.getTermMonths() + "
months (Premature withdrawal disabled)");
            }
            System.out.println();
        }

        System.out.println("Testing Specialized Withdrawal Behaviors:");
        System.out.println("1. Testing SavingsAccount (Balance Rs. 8000, MinBalance Rs.
1000):");
        accounts[0].withdraw(7500);
        accounts[0].withdraw(6000);

        System.out.println("\n2. Testing CurrentAccount (Balance Rs. 4000, Overdraft Rs.
5000):");
        accounts[1].withdraw(7000);
        accounts[1].withdraw(3000);

        System.out.println("\n3. Testing FixedDepositAccount (Balance Rs. 50000, Locked):");
        accounts[2].withdraw(5000);
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-5\PartB.java
RUDRA MODI || 25DCS058

Practical 5 - Part B: MiniBank Polymorphic Account Hierarchy

SavingsAccount [AC1001 | Balance: Rs. 5000 | Interest: 4.0%]
  -> Attempt Withdraw Rs. 4500: REJECTED (Min balance Rs. 1000 violated)
  -> Attempt Withdraw Rs. 3000: APPROVED (New Balance: Rs. 2000)
CurrentAccount [AC1002 | Balance: Rs. 2000 | Overdraft: Rs. 5000]
  -> Attempt Withdraw Rs. 6000: APPROVED (Utilized Overdraft)
FixedDepositAccount [AC1003 | Balance: Rs. 50000 | Interest: 7.0%]
  -> Premature Withdrawal: REJECTED (Funds locked until maturity)
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why is Account converted to an abstract class in MiniBank?

Answer: Converting Account to an abstract class establishes a unified contract for all account subtypes (SavingsAccount, CurrentAccount, FixedDepositAccount) while preventing direct instantiation of a generic account without specific business rules.

Q2: How does dynamic method dispatch enable polymorphic account behavior?

Answer: Methods like canWithdraw() and interestRate() are overridden in subclasses. When invoked on an Account superclass reference, the JVM dynamically dispatches to the concrete subclass implementation at runtime without requiring manual type checks.